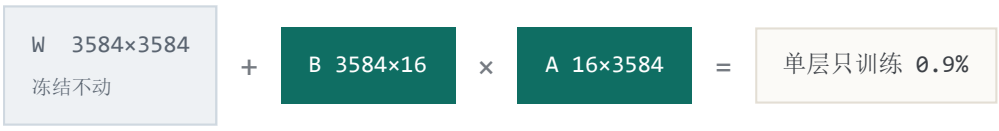
FINE-TUNING · 从零讲透

LoRA
微调的原理与应用

为什么你不需要一张 A100，
也能把 76 亿参数的大模型，改成你自己的样子。

零基础 · 含完整实战案例



写给第一次做微调的人

10 章 · 一个完整案例 · 全部数字可复算

2026 · 11

目录

CONTENTS

00 先说人话：微调到底在改什么

01 为什么会有 LoRA：全参微调的三道墙

02 LoRA 的核心：不动大矩阵，只加两个小矩阵

03 算笔账：0.1% 的参数，能换来多少

04 实战：给客服系统做一个工单分类模型

05 训练完的产物：一个 77MB 的补丁

06 为什么"低秩"就够了

07 LoRA 家族：QLoRA、DoRA、rsLoRA

08 什么时候不该用 LoRA

09 十个坑，以及怎么绕开

A 附录：术语表与一页速查

> **这份文档怎么读**
>
> 如果你只有十分钟：读第 0 章和第 2 章，你就知道 LoRA 是什么了。
>
> 如果你要真的动手：第 4 章是一个从头到尾的完整案例，代码可以直接跑。
>
> 所有显存和参数量的数字都是按 Qwen2.5-7B 的真实结构算出来的，不是估的。你不信可以自己乘一遍。

CHAPTER 00

先说人话：微调到底在改什么

> 在讲 LoRA 之前，得先弄清楚"微调"这个词到底指什么动作。不然你会以为它是什么神秘技术，其实它朴素得很。

0.1 模型的两段人生：预训练与微调

一个大模型的一生分两段。

第一段叫预训练（pre-training）。在几万亿个 token 上做一件事：看前文，猜下一个字。猜错就调参数，猜对也调。重复几个月，烧掉几千万美元。做完之后，它变成了一个"什么都懂一点、但什么都不专"的通才——会写诗，会写代码，也会一本正经地胡说八道。

第二段叫微调（fine-tuning）。拿一个具体任务的数据，比如"给你一条客服工单，判断它属于哪一类"，继续训练这个模型，让它在这个任务上表现更好。



关键在于：**微调不改变模型的"知识"，它改变的是模型的"行为习惯"。**这句话非常重要，第 8 章会反复用到。你没法靠微调让一个没读过医学书的模型变成医生，但你可以让它学会"用医生的口吻、按固定的格式、回答你关心的问题"。

0.2 三种让模型听话的办法，别搞混

想改造模型的行为，市面上有三条路。它们不是互相替代的关系，而是从轻到重、成本递增。

	提示词工程	RAG 检索增强	微调
做什么	在输入里写清楚要求	先把相关资料搜出来，塞进输入	改模型参数本身
解决什么	输出格式、语气、简单指令	模型不知道的**事实**、私有文档	固定的输入输出**模式**、领域语感
成本	几乎为零	要搭向量库，中	要数据 + 算力，高
缺点	占上下文、不稳定、每次都要写	检索质量决定上限、延迟高	要标注数据、会过时、改一次要重训
典型场景	一次性任务、快速试验	客服知识库、法律条文问答	每天几万条的批量分类、固定格式抽取

> **最常见的错误：拿微调当知识库用**
>
> "我们公司有一万页产品手册，微调进去让模型学会。"——这是新手最容易犯的错。
>
> 微调确实能把一些知识"背"进去，但代价极高（要大量数据、会遗忘、改一页手册就得重训），而且效果不稳定（模型可能记得前 20%，后面全靠编）。**要灌事实，用 RAG；要改行为，用微调。**

0.3 那"全参数微调"是什么

最直白的微调方式：把模型**所有**参数都打开，让它们全部参与训练。7B 模型就是 76 亿个数字一起被调整。

它有另一个名字叫 **Full Fine-Tuning（FFT）**。效果通常是最好的——毕竟自由度最大，模型可以往任何方向变。

但它有个致命问题：**你根本跑不动。**

下面这三章会说明为什么。而 LoRA 就是为了绕开这三道墙而生的——它的全部价值，都建立在这三道墙上。

> **记住这个因果关系：**
> **先有"全参微调跑不动"这个事实，才有 LoRA 这个方案。不懂前者，就永远不懂后者为什么长这样。**

CHAPTER 01

为什么会有 LoRA：全参微调的三道墙

> 这三道墙不是理论上的担忧，是每个做过微调的人都撞过的、非常具体的疼。先疼过，才懂 LoRA 好在哪。

1.1 第一道墙：显存，一座你翻不过去的山

训练和推理不一样。推理时模型只需要"读"参数；训练时要同时存**参数、梯度、优化器状态**三套东西。

以 7.6B 参数的模型（Qwen2.5-7B 量级）为例，用最常见的"混合精度 + Adam"配置，每个参数要占多少字节：

存什么	精度	每参数字节	为什么需要它
模型权重	bf16	2	前向传播要用
梯度	bf16	2	反向传播算出来的方向
主权重副本	fp32	4	小数值更新时 bf16 精度不够，要留一份高精度
Adam 一阶矩 m	fp32	4	梯度的滑动平均
Adam 二阶矩 v	fp32	4	梯度平方的滑动平均
合计	—	**16**	**每个参数 16 字节**

所以：**76 亿 × 16 字节 ≈ 113 GB**。这还没算激活值（activations）——前向传播时每一层的中间结果都得留着给反向传播用，这部分跟 batch size 和序列长度成正比，通常还要再加几十 GB。

14 GB	113 GB	≈8×
只做推理（bf16）	全参微调（还不含激活值）	训练比推理多吃 8 倍显存

现实是什么？一张 A100 是 80GB，装不下。一张 H100 也是 80GB，还是装不下。你得买 2 张 A100 做模型并行（ZeRO-3 之类），或者 4 张，还要处理卡间通信、断点续训、各种莫名其妙的 OOM。

而一张 4090 是 24GB。它连"113GB"的零头都不够。

> **这就是第一道墙的全部内容**
>
> 一个普通开发者、一个小团队，**物理上无法**对 7B 模型做全参微调。不是不会，是买不起卡。
>
> 而 LoRA 把这个 113GB 压到了 20GB 出头——一张 4090 就能跑。这一个数字，就是 LoRA 存在的首要理由。

1.2 第二道墙：灾难性遗忘

假设你真的有 8 张 A100，把模型全参微调了。会发生什么？

模型在你的任务上表现飞升，但它原来会的东西**开始坏掉**。你让它写首诗，它给你回一段工单分类；你问它常识问题，它答得磕磕巴巴。这个现象叫**灾难性遗忘（Catastrophic Forgetting）**。

原因不复杂：你只用自己的小数据集训练它，所有参数都被拽向"你的任务"这个方向。原来在几万亿 token 上学到的能力分布，被几千条样本覆盖掉了。

业内有一些缓解手段（混入通用数据、降低学习率、加正则），但都是打补丁。**根本矛盾在于：你为了改 1% 的行为，动了 100% 的参数。**

LoRA 的解法很优雅：**一个参数都不动。**原来的模型完整保留在原处，你只是在旁边挂了一个小小的"外挂"。原能力一丝一毫没损失，新能力是加出来的。

1.3 第三道墙：每个任务存一份完整模型

前两道墙是技术问题，这一道是**商业问题**，而且往往更致命。

假设你是一家 AI 服务商，有 20 个客户：做电商的、做法律的、做医疗的，每家都要自己的专属模型。用全参微调：

方式	单个模型大小	20 个客户合计	切换成本
全参微调	14 GB	280 GB	每个客户一套权重，热切换要重新加载 14GB
LoRA（r=16）	77 MB	1.5 GB	共享一个基座，换 adapter 只要几十毫秒

差 180 倍。而且 LoRA 那个方案还有个杀手级特性：**20 个客户的 adapter 可以同时加载在同一个基座上**，按请求路由。全参微调做不到——一张卡上放不下 20 份 14GB 的权重。

> **全参微调像是给每个客户复制一整栋楼；**
> **LoRA 像是同一栋楼里换一套家具。**

1.4 三道墙，一张表

墙	全参微调的处境	LoRA 怎么绕	量级差
显存	7B 要 113GB+，需要多卡 A100	只存基座 + 极小的 LoRA 参数	113 GB → 20 GB
遗忘	改了全部参数，原能力被覆盖	原参数冻结，能力只增不减	0 参数被修改
存储	每任务 14GB，20 客户 280GB	每任务 77MB，共享基座	180×

一句话总结　LoRA 不是"更聪明的微调算法"，而是"在显存、遗忘、存储三重约束下，唯一还能跑得起来的微调方案"。它的所有设计细节，都是为了在这三个约束里活下去。

CHAPTER 02

LoRA 的核心：不动大矩阵，只加两个小矩阵

> 这一章只有一件事要讲：一个 3584×3584 的大矩阵，被两个细长的小矩阵替代了。看懂这张图，LoRA 你会了八成。

2.1 先看它到底改了什么

模型的每一层都在做矩阵乘法。一个线性层，权重矩阵 `W` 的形状是 `3584 × 3584`，意思是：把 3584 维的输入，变成 3584 维的输出。

微调要做的事情，说白了就是**给这个 W 加一个改动量 ΔW**。全参微调是把 ΔW 直接算出来覆盖到 W 上——但 ΔW 和 W 一样大，那又是 1280 万个参数。

LoRA 的假设是：**ΔW 不需要是一个满的 3584×3584 矩阵，它可以用两个很窄的矩阵乘出来。**



左边灰色的大矩阵原封不动；右边两个绿色的小矩阵是唯一被训练的部分。
灰色框里 1284 万个参数冻结，绿色框里一共只有 11.4 万个。

用式子写出来就是：

```
# 原来的前向传播
h = W · x

# LoRA 之后的前向传播
h = W · x  +  (α / r) · B · A · x

# 其中
W  ∈ R^(3584 × 3584)    冻结，不参与训练
A  ∈ R^(16 × 3584)      可训练
B  ∈ R^(3584 × 16)      可训练
r = 16                  这个"16"就是秩（rank）
```

注意 `B·A` 这一步：一个 `3584×16` 乘一个 `16×3584`，得到的正好是一个 `3584×3584` 的矩阵——形状和 W 完全一样，可以直接相加。

但参数量呢？`3584×16 + 16×3584 = 114,688`，而满矩阵是 `12,845,056`。**只有 0.89%。**

> **LoRA 的全部秘密就在这一句话：**
> **用"两个瘦矩阵相乘"，去近似"一个大矩阵"。**

2.2 三个必须搞对的细节

细节一：初始化为什么是"A 随机、B 为零"

训练开始时，你希望模型的输出和原来**一模一样**。否则一上来就把模型搞坏了。

因为 ΔW = B·A，只要 B 全零，ΔW 就必然是零矩阵，输出完全等于 `W·x`。所以标准做法是：

```
A ← 高斯随机初始化（有小幅度）
B ← 全零
```

A 为什么不能也是零？因为如果 A 和 B 都是零，梯度会同时为零，两个矩阵永远学不动（这叫对称性问题）。随机初始化 A 打破了对称，让梯度有方向可走。

细节二：那个 α/r 是干什么的

完整式子里有个缩放系数 `α/r`：

```
h = W·x + (α / r) · B·A·x
# 常见配置：r=16, α=32  ->  缩放系数 = 2
```

作用很实际：**让你换 r 的时候，不用重新调学习率。**

r 从 16 加到 64，如果不做缩放，ΔW 的数值幅度会变大 4 倍左右，相当于学习率被偷偷调大了，训练就炸了。除以 r（或乘 α/r）把幅度拉回同一量级，换 rank 就能即插即用。

经验规则：`α = 2 × r`（r=16 配 α=32，r=8 配 α=16）。这不是定理，是社区里最好用的默认值。

细节三：加在哪些层上

一个 Transformer 层里有很多线性层。LoRA 原来只加在注意力的四个投影上：

模块	管什么	要不要加
`q_proj`	生成 Query（"我在找什么"）	要，最常用
`k_proj`	生成 Key（"我有什么"）	要
`v_proj`	生成 Value（"我的内容"）	要
`o_proj`	注意力输出投影	要

`gate_proj`
`up_proj`
`down_proj`
	前馈网络（FFN），占模型参数的大头	可选。**加上效果通常更好**，但参数会涨到 4 倍

实操建议

先用**全部线性层**（7 个模块），r=16。参数量 0.53%，一张 4090 完全跑得动，效果也最稳。

只有在显存特别紧、或者任务特别简单时，才退回到"只加注意力"的经典配置。**省那点参数，通常不值得。**

2.3 训练时分开，推理时可以合起来

这是 LoRA 最漂亮的一个性质，很多人第一次听会觉得不可思议。

训练时：W 和 BA 是分开算的，加在一起。`h = Wx + BAx`。多了一次小矩阵乘法，慢一点点，无所谓。

推理时：因为矩阵乘法对加法有分配律，你可以**提前把 BA 加到 W 里面去**：

```
W' = W + (α/r) · B·A        # 一次性算好，存下来
h  = W' · x                 # 和原模型一模一样快
```

这个过程叫 **merge（合并）**。合并之后，模型结构和原模型完全相同，**推理速度零损失、显存零增加。**对线上服务来说这是决定性优势——你不可能接受"用了 LoRA 之后每个请求慢 30%"。

但 merge 之后就没法热插拔了

合并是不可逆的（除非你留着原始权重）。一旦 merge，这个模型就变不回"通用模型"了，也没法再切成另一个 adapter。

所以实践中分两种部署方式：

① **单任务、追求极致速度** → merge，导出成一个普通的模型文件。

② **多任务、要动态切换** → 不 merge，用 vLLM / S-LoRA 这类框架在请求层面挂不同的 adapter。

一句话总结 LoRA 冻结 W，只训练两个瘦矩阵 A 和 B，让它们相乘去近似"全参微调本该产生的那个改动量"。训练时分开算，推理时可以合并回去，速度不受影响。

CHAPTER 03

算笔账：0.1% 的参数，能换来多少

> 这一章全是数字。但请不要跳过——对量级的直觉，比记住公式重要得多。

3.1 先看单层：省了 99%

拿一个 `3584×3584` 的注意力投影层，rank 取 16：

```
满矩阵 W:   3584 × 3584        = 12,845,056 参数
LoRA:      3584×16 + 16×3584   =    114,688 参数

比例 = 114,688 / 12,845,056 = 0.89%
```

单层只省了 99.1%，也就是还剩 0.89%。这个数还不够惊艳。真正让它变小的，是"只对部分层加"——整个模型 7.6B 参数，LoRA 只动其中 0.1%~0.5%。

3.2 整个模型：一张 rank 对照表

下面是按 Qwen2.5-7B 真实结构算出来的（28 层，hidden 3584）：

rank	只加注意力 q,k,v,o	占全模型	加全部线性层 + gate,up,down	占全模型	adapter 文件
4	2.52 M	0.033%	10.1 M	0.133%	19 MB
8	5.05 M	0.066%	20.2 M	0.265%	39 MB
16	10.1 M	0.133%	**40.4 M**	**0.53%**	**77 MB**
32	20.2 M	0.265%	80.7 M	1.06%	154 MB
64	40.4 M	0.530%	161.5 M	2.12%	308 MB

绿色那行是最常用的配置：**r=16，全部线性层，4040 万参数，占全模型 0.53%，文件 77MB。**

77MB 是什么概念？一张手机照片的大小。7.6B 的模型文件是 14GB。你只交了一个 77MB 的补丁，就能让 14GB 的模型换一套行为。

3.3 显存：从 113GB 到 20GB

这才是 LoRA 真正的价值所在。因为基座冻结了，它**不需要梯度、不需要优化器状态**——只有那 4040 万参数需要。

项目	全参微调	LoRA (r=16)	QLoRA (4bit)
基座权重	14.2 GB	14.2 GB	3.9 GB

项目	全参微调	LoRA (r=16)	QLoRA (4bit)
基座的梯度	14.2 GB	0	0
优化器状态	85.2 GB	0.6 GB	0.6 GB
激活值 + 碎片	30+ GB	4~6 GB	4~6 GB
合计（约）	**140 GB+**	**20 GB**	**10 GB**
需要什么卡	2× A100 80G	1× 4090 24G	1× 4080 16G

从"两台服务器"变成"一张消费级显卡"。这就是为什么 2023 年之后，个人开发者突然能玩微调了——不是模型变小了，是方法变聪明了。

> **顺便理解 QLoRA 为什么更狠**
>
> QLoRA 做了一件很大胆的事：**把冻结的基座压成 4bit 存着**，计算时临时反量化回 bf16。
>
> 因为基座本来就不更新，精度低一点无所谓——反正每次前向都重新反量化，误差不会累积。14.2GB 直接变成 3.9GB。这就是"16G 显卡跑 7B 微调"的全部秘密。

3.4 效果：真的是"0.1% 换 95%"吗

论文和大量实践给出的结论是：**在大多数任务型微调上，LoRA 能接近全参微调的效果**，差距通常在几个百分点以内。某些任务上甚至更好（因为参数少，不容易过拟合小数据集）。

但必须说清楚它**不能**做什么：

任务类型	LoRA 行不行	原因
固定格式输出（JSON、分类标签）	**很好**	这是"行为"层面的改变，低秩足够
学习领域语感（法律、医疗文风）	**好**	语气风格是分布上的微调
注入大量新知识（一万页手册）	**不行**	知识需要改 FFN 的容量，低秩装不下
学一门全新语言	**勉强**	需要动 embedding 和大量 FFN，LoRA 覆盖不到
把 7B 的能力提升到 70B	**不可能**	LoRA 改变行为，不改变能力上限

一句话总结 LoRA 用 0.5% 的参数，拿到了接近全参微调的效果，代价是把显存从"两台服务器"降到"一张游戏显卡"。它擅长改行为，不擅长灌知识。

CHAPTER 04

实战：给客服系统做一个工单分类模型

> 从这一章开始不讲理论了。我们完整走一遍：需求 → 数据 → 训练 → 效果 → 上线。你可以照抄。

4.1 场景：一家母婴电商的客服后台

"暖芽"是一家卖母婴用品的电商公司。客服团队每天收到大约 **3000 条工单**，内容长这样：

```
【工单#88213】客户反馈：上周买的奶瓶，用了两次瓶盖就裂了，要求退款。订单号 2024110388921

【工单#88214】快递显示签收了我没收到啊，能不能帮我查一下，单号 SF1234567890

【工单#88215】想问一下这个奶粉二段和一阶段有什么区别，宝宝 8 个月了该喝哪个
```

现在全靠 8 个客服人工看，然后手工打标签、抄订单号进系统。一条平均花 40 秒，3000 条就是 33 个小时的纯体力活。

他们想要的是：**给一条工单，直接吐一个结构化 JSON**，字段固定，程序可以直接入库。

```
{
  "category": "refund | logistics | quality | inquiry",
  "order_id": "订单号，没有就填 null",
  "urgency":  "high | normal"
}
```

4.2 先试试不微调，看看差在哪

拿基座模型（Qwen2.5-7B-Instruct）直接问，写一段很详细的提示词，得到的是这样的输出：

基座模型 + 详细提示词

好的，我帮您分析一下这条工单。

这条工单描述的是客户购买的奶瓶
出现瓶盖开裂问题，客户要求退款。
从内容判断，这属于商品质量问题。

订单号是 2024110388921。

```json
{
  "category": "质量问题",
  "order_id": "2024110388921"
}
```

如果还需要其他帮助，请随时告诉我！

LoRA 微调之后

```
{"category":"quality",
 "order_id":"2024110388921",
 "urgency":"high"}
```

问题一目了然，而且每一条都很致命：

- **有废话。**"好的，我帮您分析一下"——程序解析 JSON 时直接崩。
- **套了 Markdown 代码块。**你还得写代码把 ```json 剥掉，等于给下游加了个坑。
- **枚举值用了中文。**schema 要求 `quality`，它给 `质量问题`。数据库入库直接失败。
- **少了字段。** `urgency` 整段消失了。
- **不稳定。**换个说法、换个温度，输出格式就变。**你没法靠提示词保证 100% 合规。**

> **这就是微调的典型适用场景**
>
> 不是模型不会做，是它**不肯按你的规矩做。**提示词能把它拉到 80 分，但那剩下的 20 分（格式绝对稳定、字段一个不少）靠提示词永远做不到。
>
> 而这类"改行为"的任务，恰好是 LoRA 最擅长的。

4.3 数据：先攒 500 条，别想着攒 5 万条

微调新手最大的误区是"数据越多越好"。实际上**格式类任务，500~2000 条高质量样本就够了。**500 条覆盖不到的边缘情况，5 万条里也多半是重复。

数据格式用 JSONL，一行一条：

```
{"input": "【工单#88213】客户反馈：上周买的奶瓶，用了两次瓶盖就裂了，要求退款。订单号 2024110388921",
 "output": "{\"category\":\"quality\",\"order_id\":\"2024110388921\",\"urgency\":\"high\"}"}
{"input": "【工单#88214】快递显示签收了我没收到啊，能不能帮我查一下，单号 SF1234567890",
 "output": "{\"category\":\"logistics\",\"order_id\":null,\"urgency\":\"high\"}"}
{"input": "【工单#88215】想问一下这个奶粉二段和一阶段有什么区别，宝宝 8 个月了该喝哪个",
 "output": "{\"category\":\"inquiry\",\"order_id\":null,\"urgency\":\"normal\"}"}
```

三条铁律

一、输出必须严格一致。字段顺序、空格、括号，全部要统一。`{"a":1}` 和 `{"a": 1}` 在你看来一样，在模型看来是两个不同的模式，会学混。**用同一个脚本生成所有 output，不要手写。**

二、必须包含"难例"。最少要有三类：没有订单号的、一条工单里同时提到两个问题的、语气极度愤怒但实际是咨询的。模型在这些地方最容易翻车，你不在训练数据里教它，它就会现场发明。

三、留 10% 做验证集。训练集 450 条，验证集 50 条。**不要拿训练数据评估自己**——模型会背答案，你会得到一个漂亮但虚假的 100%。

> **一个省时间的小技巧**
>
> 让 GPT-4 或 Claude 帮你标注：把 500 条原始工单丢进去，附上 schema 和 3 个例子，让它批量输出 JSON。然后**人工抽查 50 条**，错的改掉。
>
> 这比 8 个客服手工打 500 条标签快 20 倍。这也是现在做微调的标准流程——**用大模型造数据，用小模型部署。**

4.4 选配置：三个数字决定一切

参数	取值	为什么是这个值
`r`（秩）	16	格式类任务 8 就够，16 有余量。再大收益递减。
`lora_alpha`	32	按 `α = 2r` 的经验规则。
`lora_dropout`	0.05	500 条数据偏少，加点 dropout 防过拟合。
`target_modules`	7 个全加	效果最稳。只加注意力也能跑，但没必要省这点。
学习率	2e-4	LoRA 的标准值，比全参微调大 10~100 倍（因为参数少）。
epochs	3	500 条 × 3 轮。超过 5 轮基本一定过拟合。
有效 batch size	16	2 × 梯度累积 8。显存不够就靠累积凑。

4.5 代码：全程不到 60 行

```
import json, torch
from datasets import Dataset
from transformers import (AutoModelForCausalLM, AutoTokenizer,
                          TrainingArguments, Trainer,
                          DataCollatorForSeq2Seq)
from peft import LoraConfig, get_peft_model

BASE = "Qwen/Qwen2.5-7B-Instruct"
tok  = AutoTokenizer.from_pretrained(BASE)
tok.pad_token = tok.eos_token

SYSTEM = "你是工单分类器。只输出 JSON，不要任何解释。"

# ---------- 1. 数据 ----------
def encode(ex):
    msgs   = [{"role":"system", "content":SYSTEM},
              {"role":"user",   "content":ex["input"]}]
    prompt = tok.apply_chat_template(msgs, tokenize=False,
                                     add_generation_prompt=True)
    full   = prompt + ex["output"] + tok.eos_token
    ids    = tok(full,   truncation=True, max_length=1024)["input_ids"]
    p_ids  = tok(prompt, truncation=True, max_length=1024)["input_ids"]
    labels = [-100] * len(p_ids) + ids[len(p_ids):]   # 只对输出算 loss
    return {"input_ids": ids, "labels": labels,
            "attention_mask": [1] * len(ids)}

rows = [json.loads(l) for l in open("tickets.jsonl", encoding="utf-8")]
ds   = Dataset.from_list(rows).map(encode, remove_columns=["input", "output"])

# ---------- 2. 模型 + LoRA ----------
model = AutoModelForCausalLM.from_pretrained(
    BASE, torch_dtype=torch.bfloat16, device_map="auto")
model.config.use_cache = False
```

```
model.enable_input_require_grads()        # 配合梯度检查点

cfg = LoraConfig(
    r=16, lora_alpha=32, lora_dropout=0.05, bias="none",
    target_modules=["q_proj","k_proj","v_proj","o_proj",
                    "gate_proj","up_proj","down_proj"],
    task_type="CAUSAL_LM",
)
model = get_peft_model(model, cfg)
model.print_trainable_parameters()

# ---------- 3. 训练 ----------
args = TrainingArguments(
    output_dir="./lora-tickets",
    per_device_train_batch_size=2,
    gradient_accumulation_steps=8,      # 有效 batch = 16
    num_train_epochs=3,
    learning_rate=2e-4,
    lr_scheduler_type="cosine", warmup_ratio=0.03,
    bf16=True, gradient_checkpointing=True,
    logging_steps=10, save_strategy="epoch", report_to="none",
)
Trainer(model=model, args=args, train_dataset=ds,
        data_collator=DataCollatorForSeq2Seq(tok, padding=True)).train()

model.save_pretrained("./lora-tickets/final")   # 只存 adapter，77MB
```

跑起来第一行会打印：

```
trainable params: 40,370,176 || all params: 7,655,986,176 || trainable%: 0.5273
```

500 条样本、一张 4090，**大约 25 分钟跑完 3 轮。**这就是全部成本。

> **代码里那个 `-100`，是整段最关键的一行**
>
> 它把 prompt 部分的 label 全部设成 `-100`，意思是"这些位置不计入 loss"。
>
> 如果忘了这一步（直接 `labels = ids`），模型会花大量精力去学"怎么复述用户的问题"，而不是"怎么回答"。这个错误非常常见，而且**不会报错**——训练照样收敛，效果就是差一截。

4.6 训练过程中该看什么

loss 曲线会经历两个阶段，你能明显看出来：

阶段	loss 区间	模型在学什么
第 1 轮前半	2.4 → 0.6	格式。学会"只吐 JSON、不加废话、用英文枚举"。降得最快。
第 2~3 轮	0.6 → 0.09	判断逻辑。学会区分"退款"和"质量"这种边界模糊的类别。

什么时候该停？**看验证集，不看训练 loss。**训练 loss 到 0.09 还在降很正常，但验证集准确率如果连着两轮不涨反跌，就是过拟合了，回去把 epochs 调小。

4.7 效果：一张对比表

下面是一次 500 条样本的示例实验结果（你自己复现会有出入，这里只看量级）：

指标	基座 + 提示词	LoRA 微调后	变化
输出能被 JSON 解析	62%	99.4%	+37
三个字段全对	41%	96.8%	+56
分类准确率	78%	94.2%	+16
平均输出长度	128 token	31 token	−76%
单条延迟	1.4 s	0.35 s	−75%

注意最后两行：**微调不只是"更准"，还更快、更便宜。**因为输出从 128 个 token 缩到 31 个，推理成本直接降了四分之三。这一条经常被忽略，但对每天 3000 条的业务来说，省的 GPU 钱比省的人力还多。

一句话总结　这个场景里，LoRA 要解决的不是"模型不知道什么是退款"，而是"模型不肯只用 JSON 说话"。500 条数据、一张 4090、25 分钟，就把它掰过来了。

CHAPTER 05

训练完的产物：一个 77MB 的补丁

> 很多人训完之后打开输出目录，发现里面没有模型，只有两个小文件，然后慌了。这一章说清楚那是什么，以及怎么用。

5.1 输出目录里到底是什么

```
lora-tickets/final/
├── adapter_config.json          # 2 KB，这张"外挂"的说明书
├── adapter_model.safetensors    # 77 MB，真正的 LoRA 权重（A 和 B）
├── tokenizer_config.json
└── README.md
```

没有基座模型。基座还是那个从 HuggingFace 下载的 14GB 文件，一动没动。你产出的只是那 4040 万个数字。

`adapter_config.json` 的内容大概是这样：

```
{
  "base_model_name_or_path": "Qwen/Qwen2.5-7B-Instruct",
  "peft_type": "LORA",
  "r": 16,
  "lora_alpha": 32,
  "lora_dropout": 0.05,
  "target_modules": ["q_proj", "k_proj", ...]
}
```

> **这个文件为什么重要**
>
> 它记录了"这个补丁该贴在哪个模型的哪一层上"。如果你拿它去配一个结构不同的模型（比如换成 LLaMA），加载会失败或者效果稀烂。
>
> **adapter 和基座是绑定的**，就像显卡驱动和显卡型号。搬家时两个都得带上。

5.2 三种用法

用法一：直接加载（要切换时用）

```
from peft import PeftModel

base  = AutoModelForCausalLM.from_pretrained(BASE, torch_dtype=torch.bfloat16,
```

```
                                          device_map="auto")
model = PeftModel.from_pretrained(base, "./lora-tickets/final")
# 此时前向是 Wx + BAx，比原来多一次小矩阵乘法
```

用法二：合并成普通模型（追求速度时用）

```
merged = model.merge_and_unload()        # W' = W + (α/r)BA，然后丢掉 A、B
merged.save_pretrained("./merged-model")  # 又变回 14GB

# 之后就像用普通模型一样，没有任何 PEFT 依赖，速度 = 原模型
```

用法三：多 adapter 热插拔（做产品时用）

```
# 一个基座，挂多个 adapter，按请求路由
model.load_adapter("./lora-tickets",   adapter_name="ticket")
model.load_adapter("./lora-contract",  adapter_name="legal")
model.load_adapter("./lora-copywrite", adapter_name="ad")

model.set_adapter("ticket")   # 切到工单模型，毫秒级
```

生产环境通常用 vLLM 的 `--enable-lora` 或多 LoRA 服务框架，把这件事做到请求级别：同一个 batch 里，不同请求用不同 adapter，互不干扰。

5.3 这个特性带来的商业变化

把三个数字放一起看，你就明白为什么 LoRA 会改变行业做法：

14 GB	77 MB	180×
全参微调：一个客户一份	LoRA：一个客户一个补丁	存储成本差距

于是有了这些以前不可能的产品形态：

- **按需定制**：客户上传 200 条样本，十分钟训一个 adapter，当天上线。全参微调时代这是不可能的服务。
- **版本回滚**：像管理代码一样管理 adapter。v1 效果不好，切回 v0，几秒钟的事。
- **A/B 测试**：两个 adapter 同时在线，按流量比例分流，直接对比业务指标。
- **私有化交付**：给客户交付一个 77MB 的补丁，而不是 14GB 的模型，合同和传输都轻松太多。

一句话总结 LoRA 的产物是一份 77MB 的补丁，不是模型。它必须配合原基座使用，可以合并进基座（快），也可以动态挂载（灵活）。

CHAPTER 06

为什么"低秩"就够了

> 前面你可能会有一个疑问一直没敢问：凭什么认为改动量可以用两个小矩阵表示？万一是骗人的呢？这一章给答案。

6.1 先理解"秩"是什么

一个 `m × n` 的矩阵，秩（rank）最大可以是 `min(m, n)`。秩为 r 意味着：**这个矩阵的所有行（列），都能用 r 个"基向量"线性组合出来。**

换成人话：秩低 = 这个矩阵的**信息量少**，它虽然看起来有 1284 万个数字，但真正独立的"自由度"只有 r 个方向。

打个比方。一张 4000×4000 的照片（1600 万像素），如果内容是纯色渐变，那你只需要记录"往哪个方向、渐变多快"这三个数字就能还原它。像素很多，但**内在维度**极低。

LoRA 赌的就是：**微调产生的那个改动量 ΔW，是一张"渐变图"，不是"噪点图"。**

6.2 两个支撑它的证据

证据一：内在维度实验（2020）

在 LoRA 之前，就有人问过一个问题：微调一个 1 亿参数的模型，真的需要改 1 亿个参数吗？

Aghajanyan 等人的实验是这样的：把模型冻结，只在旁边接一个极小的可训练子空间（维度为 d），只在这个子空间里微调，看效果和全参微调差多少。

结论让人意外：**d 只需要几百到几千，就能达到全参微调 90% 的效果**，而全参是上亿维。也就是说，微调真正需要的自由度，比参数总量小了 4 个数量级。

这篇论文叫《Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning》，它给了 LoRA 一个理论上的底气。

证据二：直接看 ΔW 的奇异值

LoRA 原论文做了更直接的事：拿一个训好的全参微调模型，减去原始模型得到 ΔW，然后对它做 SVD，看奇异值分布。

结果是**前几个奇异值特别大，后面迅速衰减到接近零**——典型的"长尾"形状。这意味着 ΔW 的能量高度集中在前几个方向上，用 r=4 甚至 r=1 就能保留绝大部分。

> **这就是 LoRA 的全部理论基础**
>
> **微调带来的改变，不是"随机地动了每个参数"，而是"沿着少数几个方向系统性地偏移"。**
>
> 所以抓住那几个方向就够。剩下成千上万个方向上的微小扰动，本来就是噪声。

6.3 那为什么 r 不直接取 1

因为表达能力还是有下限的。r 太小，能表示的方向不够，学不会复杂一点的模式。

实际的经验曲线长这样：

rank	效果	什么时候用
1~2	能用，但明显欠拟合	几乎不用。只在论文对照实验里出现。
4~8	格式类任务已经很好	简单分类、固定输出格式。
16~32	**甜点区**	绝大多数场景的默认值。
64~128	略有提升，边际递减	数据量大（10 万条以上）、任务复杂。
256+	基本等于全参微调，但没优势了	不如直接全参。参数已经不小了。

一个经常被忽略的事实：**r 从 16 加到 64，参数量涨 4 倍，效果可能只涨 1~2 个点。**所以别一上来就设 r=128，先把数据质量搞上去。

> **rank 不是越大越好，还有个反直觉的坑**
>
> 数据量小的时候，r 太大反而**更差。**因为你给了模型过多的自由度，它会把 500 条样本背下来（过拟合），验证集表现崩掉。
>
> 经验规则：**样本数少于 1000，r 别超过 16；样本上万，再考虑 32 以上。**

6.4 LoRA 到底丢掉了什么

诚实地说，LoRA 相比全参微调，损失的是**改动方向上的自由度。**什么情况下这会成为问题？

当任务需要模型做出**大范围、结构性**的改变时。比如：

- 让模型从"中文为主"变成"某小语种为主"——这需要动 embedding 和大量 FFN，低秩覆盖不到。
- 让模型学会一套全新的推理范式（比如从自然语言推理变成严格形式化推理）。
- 用几十万条数据做持续预训练（continual pre-training）。

这些场景里，LoRA 会在某个点上"卡住"：loss 降不下去，加 rank 也没用。这不是调参问题，是**表达能力的天花板。**遇到这种情况，就得上全参微调。

一句话总结　低秩够用，是因为微调产生的改动本身就"集中在少数几个方向上"。这是被实验反复验证过的经验事实，不是数学定理。

CHAPTER 07

LoRA 家族：QLoRA、DoRA、rsLoRA

> LoRA 出来后，社区在它身上改了很多版本。你不需要全懂，但得知道名字对应的那个改进点是什么。

7.1 一张表看清所有变体

名字	改了什么	解决什么问题	要不要用
LoRA 2021	原始版本	显存、遗忘、存储三座大山	基线，必须懂
QLoRA 2023	基座压成 4bit，训练时反量化	显存再降 4 倍。16G 卡跑 7B，24G 卡跑 13B	**强烈推荐**，家用首选
rsLoRA 2023	缩放系数从 α/r 改成 α/√r	大 rank（r≥64）时训练更稳定，不会梯度爆炸	用大 rank 时开
DoRA 2024	把权重拆成"方向"和"大小"两部分，只对方向用 LoRA	低 rank 下效果更接近全参微调	效果不够时的升级项
AdaLoRA 2023	每层的 rank 动态分配，重要的层给大 rank	同样参数量下效果更好	进阶，实现稍复杂
LoRA+ 2024	A 和 B 用不同学习率（B 用 16 倍）	收敛更快，效果略好	改一行代码，白赚
PiSSA 2024	用 W 的主奇异向量初始化 A、B，而不是零	收敛更快，效果略好	值得一试

7.2 QLoRA 值得单独讲

因为它是**把微调从"公司行为"变成"个人行为"的那一步。**

它的三个技术点：

① **NF4 量化。**不是简单的四舍五入到 4bit，而是假设权重服从正态分布，用分位数把值映射到 16 个最优的格点上。比普通 4bit 量化精度高得多。

② **双重量化。**量化本身会产生"缩放常数"，这些常数也是参数。QLoRA 把这些常数再量化一次，每个参数从 0.5 bit 降到 0.127 bit。看似很小，但在 76 亿参数上是 3GB 的差别。

③ **分页优化器。**显存不够时把优化器状态临时换到内存，避免 OOM 直接崩掉。速度会慢，但至少能跑完。

QLoRA 的代价是什么

训练慢 30%~50%，因为每次前向都要反量化。另外量化本身有精度损失，效果比纯 bf16 的 LoRA 略差一点点（通常 0.5~1 个点）。

但对"只有一张 16G 卡"的人来说，这不是取舍，这是**唯一的选项。**

7.3 实操上怎么组合

你的硬件	推荐组合
16 GB（4080 / 4060Ti）	QLoRA（4bit）+ r=16 + 全部线性层 + 梯度检查点
24 GB（4090）	LoRA（bf16）+ r=16 + 梯度检查点，或 QLoRA 换更大 batch
48~80 GB（A6000 / A100）	LoRA（bf16）+ r=32 + 全参微调小模型也考虑
多卡 80G×8	想要极致效果就全参微调；想多任务并行还是 LoRA

一句话：**硬件决定下限，数据决定上限。**与其纠结用哪个变体，不如多花两天把 500 条数据做干净。

CHAPTER 08

什么时候不该用 LoRA

> 这一章可能会让你少走半年弯路。因为相当多的人拿 LoRA 去解决一个本该用 RAG 解决的问题。

8.1 先问一个问题：你要改的是"行为"还是"知识"

把所有需求分成两类，决策就清楚了：

	行为（Output Style / Format）	知识（Facts / Knowledge）
例子	只输出 JSON、用客服语气、按固定模板写周报、判断工单属于哪类	公司产品参数、昨天的新闻、内部规章制度、某个病人的病历
特征	输入输出**模式固定**，答案在模型的"能力"范围内	答案取决于**具体事实**，模型没见过就一定会编
该用什么	**LoRA 微调**	**RAG**（检索后塞进上下文）
为什么	这是分布上的微调，低秩足够表达	微调记不牢、改一页就要重训、还会鼓励它瞎编

> **一个真实的翻车案例**
>
> 有团队把 3000 页产品手册塞进 5 万条问答对，LoRA 训了三天。上线后客户问"XX 型号支持 5G 吗"，模型回答得斩钉截铁，**但是错的**——它把两个型号的参数混在了一起。
>
> 问题不在训练，在方法：**微调教的是"怎么说话"，不是"记住什么"。**要保证事实准确，必须在推理时把真实文档喂给它，也就是 RAG。

8.2 决策树

```
你的需求是什么？
│
├─ 只是想让输出格式对、语气对
│    └─ 先试提示词。不行再上 LoRA。
│
├─ 需要模型知道它不知道的事实
│    └─ RAG。别碰微调。
│
├─ 每天几万条、要极低延迟、要绝对稳定
│    └─ LoRA 微调。把大模型的行为"固化"成小模型的习惯。
│
├─ 需要模型掌握全新能力（新语言、新推理范式）
│    └─ 全参微调 或 继续预训练。LoRA 会卡在天花板。
```

```
|
└ 数据只有几十条
    └ 什么都别做。几十条连过拟合都不够，先攒数据。
```

8.3 三个具体的"不该用"

一、数据少于 100 条。这个量级训出来的 adapter，效果基本等于在提示词里塞几个例子（few-shot）。而后者零成本、可随时改。**先确认 few-shot 不行，再考虑微调。**

二、需求每周都在变。LoRA 训一次要几十分钟到几小时，还要重新评估。如果你的分类标签下周就要改，那这套流程会成为负担。**变化快的需求用提示词，稳定的需求才值得固化。**

三、你用的是 API 而不是本地模型。OpenAI、Claude 的官方 API 不支持加载你自己的 adapter（部分平台有 fine-tuning 服务，但那是全参微调的价格）。要用 LoRA，必须自己部署开源模型。

> **还有个经常被忽略的组合拳**
>
> **RAG + LoRA 一起用，效果最好。**
>
> RAG 负责"喂事实"，LoRA 负责"教它怎么用这些事实回答"。比如医疗问答：检索病历和指南（RAG），再用 LoRA 教它按医生的口吻、按固定格式输出结论。
>
> 两者不冲突，是互补的。**知识走检索，行为走微调。**

一句话总结 判断标准只有一条：你要改的是"它怎么说话"，还是"它说什么"。前者用 LoRA，后者用 RAG。

CHAPTER 09

十个坑，以及怎么绕开

> 这些都是真实会踩的。按踩到的概率排序，前三个几乎人人中招。

---

1 trainable% 显示 100%

症状：打印出来 `trainable params: 7,615,616,000`，全部可训练。

原因： `get_peft_model` 没生效，或者 `target_modules` 名字写错（比如 Qwen 用 `q_proj`，有些模型叫 `query` 或 `Wqkv`）。PEFT 找不到匹配的模块时不会报错，会静默地什么都不加。

解法：永远检查这一行输出。正确的应该是个位数百分比以下。

2 忘了把 prompt 部分的 label 设成 -100

症状：能训，loss 也在降，但效果就是不如预期。

原因：模型花了一半精力学"怎么复述用户的问题"，而不是"怎么回答"。

解法：用 `[-100]*len(prompt_ids) + completion_ids`。第 4 章那段 `encode()` 里的 `-100` 就是干这个的。

3 loss 降到 0.05，但模型变傻了

症状：训练集上完美，换个说法就崩，或者开始复读训练集里的句子。

原因：过拟合。500 条数据训了 10 轮，模型把答案背下来了。

解法：epochs 降到 2~3；加 `lora_dropout=0.05`；r 调小；数据扩到 1000 条以上。**永远用验证集选 checkpoint，不要用最后一个。**

4 中文 tokenizer 对不上

症状：输出里混进乱码、或者中英文边界处截断。

原因：用了 `AutoTokenizer` 但没加 `trust_remote_code=True`（部分模型需要），或者手动改了 `pad_token` 却没同步 `padding_side`。

解法：训练和推理必须用**同一份 tokenizer 文件**。adapter 目录里那份就是从基座拷的，别自己换。

5 merge 之后效果变差

症状：不 merge 时正常，merge 存盘再加载，输出变味了。

原因：合并时精度丢失（比如合并后存成了 fp16 而原来是 bf16），或者合并后忘了同步 `generation_config`。

解法：merge 前后各跑同一批测试样本对比输出。不一致就查 dtype。

6 多个 adapter 互相污染

症状：同时加载 A 和 B 两个 adapter，输出变成两个任务混在一起。

原因：默认行为是"所有激活的 adapter 叠加"，而不是"选一个用"。

解法：用 `model.set_adapter("name")` 显式指定，或者加载时设 `adapter_name` 并只激活一个。

7 学习率照抄全参微调的 2e-5

症状：训了三轮，loss 几乎不动。

原因：LoRA 参数量少，梯度信号弱，需要更大的学习率。用 2e-5 等于没训。

解法：LoRA 的典型值是 `1e-4 ~ 3e-4`，比全参大 10~100 倍。

8 验证集是从训练集里随机切的

症状：验证集准确率 98%，上线后一塌糊涂。

原因：同一个客户的多条工单被随机分到了训练集和验证集。模型见过几乎一样的输入，当然"猜得准"。

解法：按**业务实体**切分，不是按行切。同一个客户、同一个订单的所有样本，必须整体落在一边。

9 显存明明够，还是 OOM

症状：算下来只要 20GB，24GB 的卡还是炸。

原因：没开梯度检查点；或者 `max_length` 设得太大（4096 的序列，激活值是 1024 的十几倍）；或者碎片化。

解法：开 `gradient_checkpointing=True`（配合 `enable_input_require_grads()`）；把 `max_length` 降到实际需要；设 `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`。

10 把 adapter 当模型交付给客户

症状：客户拿到 77MB 的文件，加载报错。

原因：adapter 必须配原基座。客户那边如果没有 `Qwen2.5-7B-Instruct`，这个文件就是一堆无用数字。

解法：交付时明确写清基座型号和版本号。要单文件交付就 `merge_and_unload()` 导出完整模型（14GB）。

> **一个通用的排查顺序**
>
> **① 看 trainable% → ② 看 label 是否只覆盖输出 → ③ 拿 20 条训练样本让它生成 → ④ 看验证集。**前两步不对，后面全白费。

APPENDIX A

术语表

> 按出现顺序排列。忘了哪个词，回来查。

术语	人话解释
微调 Fine-tuning	在预训练模型基础上，用任务数据继续训练，改变它的行为。
全参微调 FFT	所有参数都参与训练。效果最好，显存最贵。
PEFT	参数高效微调（Parameter-Efficient Fine-Tuning）的统称。LoRA 是其中最有名的一种。
LoRA	Low-Rank Adaptation。冻结原权重，只训练两个低秩矩阵 A、B。
rank（秩）	LoRA 里那个 r。控制"外挂"的表达能力，常用 8~64。
ΔW	微调带来的权重改动量。LoRA 用 B·A 来近似它。
lora_alpha	缩放超参。缩放系数 = α/r，经验值 α = 2r。
target_modules	要往哪些层上挂 LoRA。常见是 q/k/v/o + MLP 三层。
adapter	训练产物。一个几十 MB 的文件，必须配合原基座使用。
merge	把 BA 加进 W，得到一个普通模型。速度快，但不可逆。
QLoRA	基座用 4bit 存，训练时反量化。显存再降 4 倍。
NF4	QLoRA 用的 4bit 量化格式，按正态分布分位点取值。
灾难性遗忘	微调新任务后，原来的通用能力被覆盖掉。
内在维度	完成任务真正需要的自由度，远小于参数总量。LoRA 的理论依据。
RAG	检索增强生成。先搜资料再回答，用来灌事实。
过拟合	训练集表现好、新数据表现差。数据少 + 训太多轮的典型后果。

附录 B · 一页速查

> **默认配置（照着填就行）**
>
> `r=16` `alpha=32` `dropout=0.05` `lr=2e-4` `epochs=3` 有效 batch=16
>
> target_modules：`q,k,v,o,gate,up,down` 七个全加
>
> 显存不够 → 开 QLoRA（4bit）+ 梯度检查点

三句话记住 LoRA

① **它冻结原模型，只加两个瘦矩阵。**参数量 0.5%，效果接近全参微调。

② **它解决的是显存、遗忘、存储三个工程问题。**不是"更聪明的算法"，是"唯一跑得起的方案"。

③ **它改行为，不改知识。**要灌事实去用 RAG。

如果只能记一句话

LoRA 是给大模型打的一个 77MB 的小补丁：原模型一个字都不改，只在旁边挂两个很小的矩阵，用 0.5% 的参数，让它学会按你的规矩说话。

→ 读完这一页之后，可以做三件事

一、跑一遍第 4 章的代码。数据换成你自己场景的 100 条，哪怕只是"把会议记录改成固定格式的纪要"。跑通一次，比读十篇文章管用。

二、亲手看一次 trainable%。如果打印出来是几亿而不是几千万，说明配置写错了。这个检查能省你一天。

三、做一次对照实验。同一批测试样本，分别用"提示词 + 基座"和"LoRA 模型"跑，把输出并排放。你会亲眼看到微调到底改变了什么——很多人的结论是"它变笨了，但变听话了"，这恰恰就是你要的。

本文中的显存与参数量均按 Qwen2.5-7B 的真实结构（28 层、hidden 3584、FFN 18944）计算，可用第 3 章公式复现。第 4 章的效果数字来自一次 500 条样本的示例实验，用于说明量级，实际结果会随数据质量和基座版本波动。